

LAPORAN PRATIKUM
ALGORITMA DAN PEMROGRAMAN
STACK (TUMPUKAN)

Disusun Oleh:

Khairun Nisa

2511532015

Dosen Pengampu:

DR. WAHYUDI, S.T, M.T

Asisten Pratikum:

Sherly Sukmadira



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas terselesaikannya laporan praktikum Algoritma dan Pemrograman ini. Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilaksanakan pada tanggal 7 April 2026. Melalui laporan ini, penulis berharap dapat memberikan gambaran yang jelas mengenai STACK (TUMPUKAN).

Penulis mengucapkan terima kasih kepada Bapak Dosen serta asisten laboratorium yang telah membimbing selama praktikum berlangsung. Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari kata sempurna, baik dari segi penyusunan, bahasa, maupun kedalaman materi. Oleh karena itu, penulis dengan rendah hati menerima segala kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi untuk perkembangan ilmu pengetahuan di teknologi informasi.

Padang, 17 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum	1
1.3 Manfaat Pratikum	2
BAB II PEMBAHASAN	3
2.1 StackArray	3
2.2 StackArrayDriver	5
2.3 NilaiMaksimum	7
2.4 SiswaStack	9
2.5 StackPostfix	11
BAB III KESIMPULAN	14
3.1 Kesimpulan	14
DAFTAR PUSTAKA	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak dan ilmu komputer, struktur data merupakan fondasi penting yang menentukan efisiensi penyimpanan, pengolahan, dan pengembalian data. Pemilihan struktur data yang tepat tidak hanya memengaruhi kompleksitas waktu dan ruang penyimpanan algoritma, tetapi juga berdampak pada kemudahan implementasi serta skalabilitas program. Salah satu struktur data linear yang paling fundamental dan sering diaplikasikan adalah *stack* (tumpukan).

Stack adalah struktur data linear yang digunakan untuk menyimpan sekumpulan objek atau variabel. Setiap penambahan (*push*) dan penghapusan data (*pop*) terjadi di ujung atas tumpukan. Sifat khas stack adalah LIFO (Last In First Out) data terakhir yang masuk akan menjadi data pertama yang keluar [1].

Struktur data stack menawarkan efisiensi tinggi berkat prinsip LIFO yang memungkinkan operasi dasar berjalan cepat dengan implementasi sederhana serta manajemen memori yang otomatis dan terprediksi. Namun, kapasitasnya yang terbatas rentan memicu overflow, dan akses yang hanya dibatasi pada elemen teratas membuatnya tidak fleksibel untuk kebutuhan penelusuran acak atau penyimpanan data berskala besar [2].

Berdasarkan karakteristik serta kelebihan dan kekurangan tersebut, pemahaman teoritis mengenai stack perlu diperkuat melalui implementasi praktis untuk menguji mekanisme kerjanya dalam skenario pemrograman nyata.

1.2 Tujuan Praktikum

1. Mahasiswa mampu menjelaskan prinsip kerja struktur data stack dengan mekanisme Last In First Out (LIFO) serta mengidentifikasi operasi fundamentalnya seperti *push*, *pop*, *peek*, *isEmpty*, dan *isFull*.
2. Mahasiswa mampu merancang dan menulis kode program untuk struktur data stack menggunakan array statis (*stackArray*) tanpa

bergantung pada panduan langkah-demi-langkah, termasuk mengelola indeks dan menangani kondisi overflow maupun underflow.

3. Mahasiswa mampu memanfaatkan struktru stack untuk menyelesaikan permasalahan komputasi spesifik.

1.3 Manfaat Praktikum

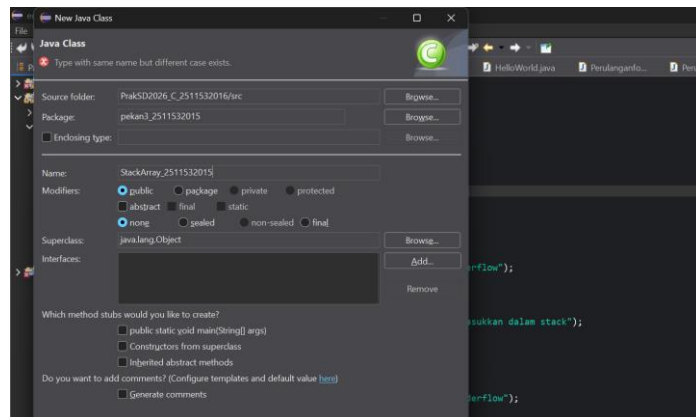
1. Memberikan pengalaman langsung dalam memvisualisasikan aliran data LIFO, sehingga konsep abstrak struktur data menjadi lebih konkret dan mudah dipahami.
2. Melatih kemampuan coding, debugging, dan perancangan algoritma.
3. Membentuk kebiasaan berpikir sistematis dalam memecahkan masalah, terutama yang berkaitan dengan urutan eksekusi dan manajemen memori.

BAB II

PEMBAHASAN

2.1 StackArray

2.1.1 Buat package pekan3_2511532015 lalu buat class baru dengan nama StackArray_2015.



Gambar 2.1.1: Pembuatan package dan Class

2.1.2 Ketikkan kode program stackArray_2511532015

```
package pekan3_2511532015;

public class stackArray_2511532015 {
    static final int MAX_2015 = 1000;
    int top_2015;
    int a_2015[] = new int [MAX_2015];
    boolean isEmpty()
    {
        return (top_2015 < 0);
    }
    stackArray_2511532015()
    {
        top_2015 = -1;
    }
    boolean push_2015 (int x)
    {
        if (top_2015 >= (MAX_2015 - 1)) {
            System.out.println ("Stack Overflow");
            return false;
        }
        else {
            a_2015[++top_2015] = x;
            System.out.println (x + " dimasukkan dalam
stack");
            return false;
        }
    }
    int pop_2015()
    {

```

```

        if (top_2015 < 0) {
            System.out.println ("Stack Underflow");
            return 0;
        }
        else {
            int x = a_2015 [ top_2015--];
            return x;
        }
    }
    int peek_2015()
    {
        if (top_2015 < 0) {
            System.out.println ("Stack Underflow");
            return 0;
        }
        else {
            int x = a_2015 [top_2015];
            return x;
        }
    }
    void print_2015 () {
        for ( int i = top_2015; i > -1; i--) {
            System.out.print (" "+ a_2015[i]);
        }
    }
}

```

2.1.3 Atribut dan Konstanta Kapasitas

- Pada deklarasi variable Class, digunakan konstanta static final int MAX_2015 = 1000 untuk menetapkan batas maksimal elemen secara tetap
- top_2015: berperan sebagai penunjuk indeks elemen teratas
- Array a_2015[]: sebagai wadah penyimpanan
- Static final sebagai ukuran stack yang tidak dapat diubah secara dinamis

2.1.4 Inisialisasi objek melalui konstruktor

Konstruktor `stackArray_2511532015()` dipanggil saat instalasi objek. Di dalamnya terdapat `top_2015` diinisialisasikan ke -1, yang merupakan konversi standar untuk menandakan stack dalam keadaan kosong.

2.1.5 Operasi manipulasi data

- isEmpty(): mengembalikan nilai true jika top_2015 kurang dari 0, digunakan sebagai guard condition sebelum operasi pengambilan atau pembacaan data.
- Push_2015(int x): menambahkan elemen ke puncak teratas stack, sebelumnya dilakukan pengecekan top_2015 >= MAX_2015 -1 jika aman, indeks top_2015 dinaikkan terlebih dahulu dengan ++top_2015, kemudian nilai x disimpan.
- Pop_2015(): menghapus dan mengembalikan elemen teratas. Dilakukan pengecekan underflow terlebih dahulu. Jika stack tidak kosong, nilai pada a_2015[top_2015] disimpan sementara, top_2015 diturunkan (top_2015--), dan nilai dikembalikan. Proses ini menjamin prinsip LIFO tetap terjaga.
- Peek_2015(): Mirip dengan pop, tetapi tidak mengubah nilai top_2015. Berguna untuk melihat elemen teratas tanpa mengganggu struktur stack.

2.1.6 Method print_2015() melakukan iterasi menurun dari top_2015 hingga 0 menggunakan perulangan for. Pendekatan ini memastikan elemen dicetak dari posisi teratas hingga terbawah, sehingga output yang dihasilkan selaras dengan representasi visual stack secara logis.

2.2 StackArrayDriver

```
package pekan3_2511532015;

public class stackArrayDriver_2511532015 {

    public static void main(String[] args) {
        stackArray_2511532015 s_2015 = new stackArray_2511532015();
        s_2015.push_2015 (10);
        s_2015.push_2015 (20);
        s_2015.push_2015 (30);
        System.out.println (s_2015.pop_2015()+ " dikeluarkan dari stack");
        System.out.println ("Elemen teratas adalah : "+ s_2015.peek_2015());
    }
}
```

```

        System.out.print ("Element pada stack : ");
        s_2015.print_2015();
    }
}

```

Program ini berfungsi sebagai modul pengujian yang dirancang untuk memvalidasi kinerja kelas `stackArray_2511532015`.

2.2.1 Baris `stackArray_2511532015 s_2015 = new stackArray_2511532015();` melakukan instansi objek. Secara otomatis, konstruktor kelas `stack` dipanggil untuk menginisialisasi `top_2015 = -1` dan mengalokasikan array `a_2015` berkapasitas 1000 elemen, di sini `stack` dalam keadaan kosong dan siap menerima operasi.

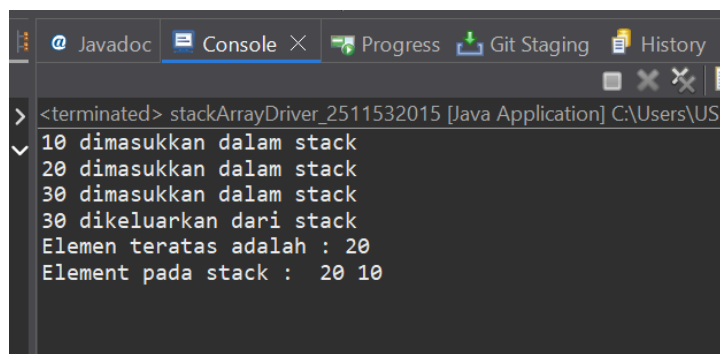
2.2.2 Selanjutnya, program mensimulasikan penyisipan atau penambahan tiga elemen secara berurutan menggunakan method `push_2015()`. Sesuai prinsip LIFO, elemen yang pertama dimasukkan akan berada pada tumpukan terakhir.

2.2.3 Method `pop_2015()` digunakan untuk mengeluarkan elemen teratas.

2.2.4 Sedangkan method `peek_2015()` digunakan untuk membaca elemen teratas setelah tadi di hapus (`pop`) tanpa melakukan penghapusan.

2.2.5 Method `print_2015` untuk menampilkan seluruh sisa elemen yang tersimpan yang dicetak dari posisi teratas menuju terbawah.

2.2.6 Ketika program dijalankan, akan menghasilkan output:



```

> <terminated> stackArrayDriver_2511532015 [Java Application] C:\Users\USI
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah : 20
Element pada stack : 20 10

```

Gambar 2.2.1: Hasil output `stackArrayDriver_2511532015`

Berdasarkan hasil output tersebut, dapat dilihat method `push_2015()` berhasil menambahkan angka 10, 20, dan 30 ke dalam `stack` secara

berurutan. Menerapkan prinsip LIFO angka 30 akan berada di paling atas dan 10 terbawah. Setelah penghapusan menggunakan method `pop_2015()` yang mengeluarkan angka teratas yaitu 30, method `peek_2015()` akan mengonfirmasi bahwa elemen teratas kini adalah 20. Terakhir, method `print_2015()` menampilkan sisa elemen yang dicetak dari posisi teratas ke terbawah.

2.3 NilaiMaksimum_2511532015

```
package pekan3_2511532015;

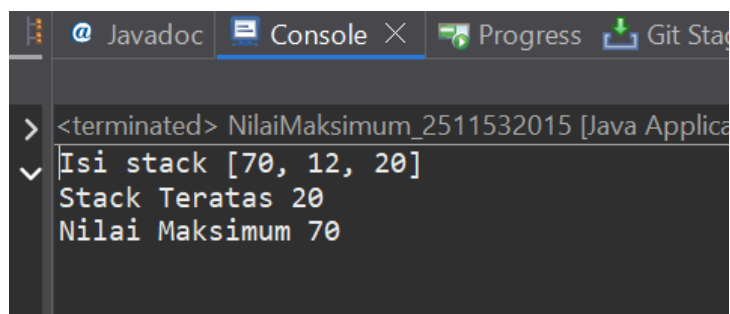
import java.util.Stack;

public class NilaiMaksimum_2511532015 {
    public static int max (Stack<Integer> s_2015) {
        Stack<Integer> backup_2015 = new Stack<Integer>();
        int maxValue_2015 = s_2015.pop();
        backup_2015.push(maxValue_2015);
        while (!s_2015.isEmpty()) {
            int next_2015 = s_2015.pop();
            backup_2015.push(next_2015);
            maxValue_2015 = Math.max(maxValue_2015,
next_2015);
        }
        while (!backup_2015.isEmpty()) {
            s_2015.push(backup_2015.pop());
        }
        return maxValue_2015;
    }
    public static void main (String[] args) {
        Stack<Integer> s_2015 = new Stack<Integer>();
        s_2015.push(70);
        s_2015.push(12);
        s_2015.push(20);
        System.out.println ("Isi stack "+ s_2015);
        System.out.println ("Stack Teratas "+
s_2015.peek());
        System.out.println ("Nilai Maksimum "+max(s_2015));
    }
}
```

Program ini dirancang untuk mencari nilai tertinggi dalam struktur data stack, karena stack menerapkan prinsip LIFO program ini menggunakan stack sementara (`backup_2015`) untuk menelusuri seluruh elemen tanpa merusak urutan asli. Dengan demikian, nilai maksimum dapat ditemukan

secara akurat sambil menjaga integritas data stack tetap utuh setelah ioperasi selesai.

- 2.3.1 Kelas NilaiMaksimum_2511532015 membuat metode statis max yang menerima parameter stack integer, metode ini dirancang untuk mencari nilai tertinggi tanpa mengubah struktur akhir stack. Stack sementara backup_2015 digunakan sebagai penyimpanan cadangan selama proses pencarian berlangsung.
- 2.3.2 Program mengambil elemen teratas dari stack utama menggunakan pop() dan menjadikannya sebagai nilai maksimum awal (maxValue_2015) dan disimpan langsung ke backup_2015.
- 2.3.3 Perulangan while pertama berjalan selama stack utama tidak kosong. Pada setiap iterasi, elemen teratas diambil lalu disimpan ke backup_2015 lalu dibandingkan dengan maxValue_2015 menggunakan fungsi Math.max() dimana nilai yang lebih besar akan menjadi nilai maksimum baru.
- 2.3.4 Perulangan while kedua memindahkan kembali seluruh elemen dari backup_2015 ke stack utama.
- 2.3.5 Metode max mengembalikan nilai maxValue_2015 yang telah melalui proses perbandingan dengan seluruh elemen stack.
- 2.3.6 Untuk main method, objek stack baru dibuat dan diisi dengan tiga nilai yaitu 70, 12, dan 20. Program kemudian mencetak isi stack dan menampilkan elemen teratas melalui peek() dan memanggil metode max() untuk mendapatkan nilai maksimum.



```
<terminated> NilaiMaksimum_2511532015 [Java Applica
Isi stack [70, 12, 20]
Stack Teratas 20
Nilai Maksimum 70
```

Gambar 2.3.1: Hasil output program NilaiMaksimum_2511532015

2.4 SiswaStack

```
package pekan3_2511532015;

import java.util.ArrayList;

class Siswa_2511532015{
    String nama_2015;
    int nim_2015;

    public Siswa_2511532015 (String nama_2015, int nim_2015) {
        this.nama_2015 = nama_2015;
        this.nim_2015 = nim_2015;
    }

    @Override
    public String toString () {
        return "Nim: " + nim_2015 + ", Nama: " + nama_2015;
    }
}

public class SiswaStack_2511532015 {
    private ArrayList<Siswa_2511532015> stack;

    public SiswaStack_2511532015() {
        stack = new ArrayList<>();
    }
    public void push_2015 (Siswa_2511532015 mhs) {
        stack.add(mhs);
    }
    public Siswa_2511532015 pop_2015() {
        if (!isEmpty_2015()) {
            return stack.remove(stack.size()-1);
        }
        return null;
    }
    public Siswa_2511532015 peek_2015() {
        if (!isEmpty_2015()) {
            return stack.get(stack.size()-1);
        }
        return null;
    }
    public boolean isEmpty_2015() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa_2015() {
        for (int i_2015 = stack.size()-1; i_2015 >=0; i_2015--) {
            System.out.println (stack.get(i_2015));
        }
    }
    public static void main(String[] args) {
        SiswaStack_2511532015 studentStack= new
        SiswaStack_2511532015();

        Siswa_2511532015 mhs1_2015 = new Siswa_2511532015 ("Ali",
        1);
```

```

        Siswa_2511532015 mhs2_2015 = new Siswa_2511532015 ("Boby",
2);
        Siswa_2511532015 mhs3_2015 = new Siswa_2511532015
("Charles", 3);
        studentStack.push_2015(mhs1_2015);
        studentStack.push_2015(mhs2_2015);
        studentStack.push_2015(mhs3_2015);

        System.out.println ("Siswa di dalam stack: ");
        studentStack.tampilkanSiswa_2015();

        System.out.println ("Siswa teratas " +
studentStack.peek_2015());
        System.out.println ("Mengelurkan siswa teratas dari stack:
"+ studentStack.pop_2015());
        System.out.println ("Daftar siswa setelah di pop: ");
        studentStack.tampilkanSiswa_2015();
    }
}

```

Program ini untuk mengimplementasikan struktur data stack yang menyimpan objek bertipe data custom, bukan hanya tipe primitif seperti int atau char. Berbeda dengan stack berbasis array statis yang memiliki kapasitas tetap, program ini memanfaatkan ArrayList sehingga ukuran stack bersifat dinamis dan dapat menyesuaikan diri secara otomatis saat data ditambahkan atau dihapus. Pendekatan ini lebih aman dari risiko overflow dan lebih relevan untuk kebutuhan manajemen data nyata.

- 2.4.1 Berfungsi sebagai cetak biru objek mahasiswa. Kelas ini memiliki dua atribut utama (nama_2511532015 dan nim_2015), sebuah konstruktor untuk mengisi nilai awal, serta metode toString() yang dioverride. Override ini memungkinkan objek mahasiswa dicetak langsung ke konsol dalam format yang rapi tanpa perlu memanggil atribut satu per satu.
- 2.4.2 Variabel private ArrayList<Siswa_2511532015> stack; menjadi wadah penyimpanan utama. Konstruksi stack = new ArrayList<>(); di dalam konstruktor kelas memastikan wadah siap digunakan saat objek SiswaStack dibuat.
- 2.4.3 Menambahkan objek mahasiswa ke dalam stack menggunakan stack.add(mhs). Karena ArrayList menambahkan elemen di indeks terakhir secara default, elemen yang baru masuk secara logika selalu berada di posisi teratas stack, sesuai prinsip LIFO.

- 2.4.4 Pop_2015() menggunakan remove() untuk mengambil sekaligus menghapus elemen teratas, sedangkan peek_2015() menggunakan get() hanya untuk membaca nilai tanpa menghapus. Keduanya di cek isEmpty_2015() untuk mencegah error, dan mengembalikan null jika stack sedang kosong.
- 2.4.5 Melakukan perulangan menurun dari indeks terakhir ke 0. Langkah ini sengaja dilakukan agar output dicetak dari elemen teratas menuju terbawah, sehingga tampilan konsol selaras dengan representasi visual stack secara logika.
- 2.4.6 Program diuji dengan membuat tiga objek mahasiswa (Ali, Bobby, Charles), lalu memasukkannya ke stack secara berurutan. Setelah itu, program mencetak seluruh isi stack, menampilkan elemen teratas melalui peek, menghapus elemen teratas dengan pop, dan mencetak ulang sisa data.

```

<terminated> SiswaStack_2511532015 [Java Application] C:\Users\USER\AppData\Local\Temp\org.eclipse.justj.openj
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Bobby
Nim: 1, Nama: Ali
Siswa teratas Nim: 3, Nama: Charles
Mengelurkan siswa teratas dari stack: Nim: 3, Nama: Charles
Daftar siswa setelah di pop:
Nim: 2, Nama: Bobby
Nim: 1, Nama: Ali

```

Gambar 2.4.1: Hasil Output program SiswaStack_2511532015

2.5 StackPostfix_2511532015

```

package pekan3_2511532015;

import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511532015 {
    public static int postfixEvaluate (String expression) {
        Stack<Integer> s_2015 = new Stack<Integer>();
        Scanner input_2015 = new Scanner(expression);
        while(input_2015.hasNext()) {
            if (input_2015.hasNextInt()) {
                // an operand (integer)
                s_2015.push(input_2015.nextInt());
            } else {
                // an operator
            }
        }
    }
}

```

```

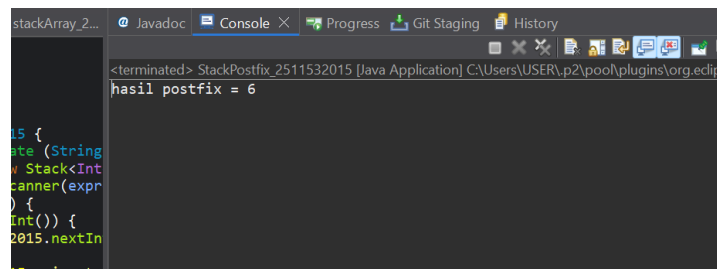
        String operator_2015 =
input_2015.next();
        int operand2_2015 = s_2015.pop();
        int operand1_2015 = s_2015.pop();
        if (operator_2015.equals("+")) {
            s_2015.push (operand1_2015 +
operand2_2015);
        } else if (operator_2015.equals("-")) {
            s_2015.push(operand1_2015 -
operand2_2015);
        } else if (operator_2015.equals("*")) {
            s_2015.push(operand1_2015 *
operand2_2015);
        } else {
            s_2015.push(operand1_2015/operand2_2015);
        }
    }
    input_2015.close();
    return s_2015.pop();
}
public static void main (String[] args) {
    System.out.println ("hasil postfix = "+
postfixEvaluate ("5 2 4 * + 7 -"));
}
}

```

Program StackPostfix_2511532015 dirancang untuk menghitung hasil ekspresi matematika yang ditulis dalam notasi postfix (juga disebut Reverse Polish Notation). Pada notasi ini, operator ditulis setelah operand-nya.

- 2.5.1 Metode postfixEvaluate menerima string ekspresi sebagai input. Stack s_2015 dibuat untuk menyimpan angka sementara, dan Scanner digunakan untuk memotong-motong string input menjadi token (angka atau operator) yang mudah diproses.
- 2.5.2 Pada perulangan while, if (input_2015.hasNextInt()) jika token adalah angka, langsung dimasukkan ke dalam stack, else jika token adalah operator, program mengambil dua angka teratas dari stack, menghitung hasilnya, lalu menyimpan kembali hasilnya ke stack.
- 2.5.3 Setelah perulangan selesai, stack hanya akan menyisakan satu elemen, yaitu hasil akhir. Metode mengembalikan nilai ini melalui return s_2015.pop().

2.5.4 Berdasarkan hasil eksekusi program, konsol menampilkan keluaran hasil postfix = 6. Nilai ini merupakan hasil akhir dari perhitungan ekspresi notasi postfix "5 2 4 * + 7 -" yang diproses secara berurutan oleh program. Secara rinci, program pertama-tama menghitung $2 * 4$ sehingga menghasilkan 8, kemudian menjumlahkan $5 + 8$ menjadi 13, dan terakhir mengurangi $13 - 7$ yang menghasilkan nilai akhir 6.



```
StackArray_2... Javadoc Console X Progress Git Staging History
<terminated> StackPostfix_2511532015 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse
hasil postfix = 6

15 {
    evaluate (String
    Stack<Int>
    Scanner(expr
    ) {
    Int()) {
    2015.nextIn
```

Gambar 2.5.1: Hasil output program StackPostfix

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan seluruh implementasi program yang telah dilakukan, dapat disimpulkan bahwa struktur data stack merupakan struktur data linear yang beroperasi berdasarkan prinsip Last In First Out (LIFO), di mana elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Stack dapat diimplementasikan menggunakan array statis dengan kapasitas tetap maupun ArrayList yang bersifat dinamis. Operasi dasar stack meliputi push untuk menambahkan elemen, pop untuk menghapus dan mengambil elemen teratas, serta peek untuk melihat elemen teratas tanpa menghapusnya. Penerapan stack sangat luas, mulai dari pencarian nilai maksimum, pengelolaan data objek terstruktur seperti data siswa, hingga evaluasi ekspresi matematika notasi postfix. Meskipun memiliki keterbatasan akses yang hanya berfokus pada elemen teratas, stack terbukti efisien dan andal dalam menyelesaikan berbagai permasalahan komputasi dengan kompleksitas waktu yang optimal.

DAFTAR PUSTAKA

- [1] Prasatya, "Apa Itu Stack dalam Ilmu Pemrograman," CODEPOLITAN, 4 June 2025. [Online]. Available: <https://www.codepolitan.com/blog/apa-itu-stack-dalam-ilmu-pemrograman/>. [Accessed 17 April 2026].
- [2] Jobstreet, "Apa Itu Stack? Ini Arti, Kelebihan, dan Kekurangannya," Jobstreet, 20 May 2024. [Online]. Available: <https://id.jobstreet.com/id/career-advice/article/stack-adalah>. [Accessed 18 April 2026].

LAPORAN TUGAS PEKAN 3

1. Penjelasan mengenai kelas ADT Website_2511532015

```
package pekan3_2511532015;

public class Website_2511532015 {
    private String judul_2015;
    private String url_2015;

    public Website_2511532015(String judulInput, String
urlInput) {
        this.judul_2015 = judulInput;
        this.url_2015 = urlInput;
    }
    public String getJudul_2015() {
        return judul_2015;
    }
    public String getUrl_2015() {
        return url_2015;
    }
    public void setJudul_2015(String judulInput) {
        this.judul_2015 = judulInput;
    }
    public void setUrl_2015(String urlInput) {
        this.url_2015 = urlInput;
    }
    @Override
    public String toString() {
        return "Judul: " + judul_2015 + "\n" +
            "URL    : " + url_2015;
    }
}
```

Kelas Website berfungsi sebagai Abstract Data Type (ADT) atau cetak biru yang mendefinisikan struktur data untuk merepresentasikan entitas halaman web dalam simulasi riwayat browser. Kelas ini memiliki dua atribut inti, yaitu judul dan URL. Akses terhadap data dikelola secara terkontrol melalui method getter dan setter, yang memungkinkan kelas driver membaca atau memperbarui informasi halaman saat operasi. Method toString() juga telah di-override untuk mengembalikan representasi teks yang terformat rapi dalam baris terpisah.

2. Penjelasan mengenai kelas Browser_2511532015

```
package pekan3_2511532015;

import java.util.Scanner;
import java.util.Stack;
```

```

public class Browser_2511532015 {
    //menampilkan menu
    public static void tampilkanMenu_2511532015() {
        System.out.println ("\n=== Browser History NIM: 2511532015
        ===");
        System.out.println("1. Kunjungi Website (Push)");
        System.out.println("2. Tombol Back (Pop)");
        System.out.println("3. Lihat Halaman Aktif (Peek)");
        System.out.println("4. Cek Status History");
        System.out.println("5. Keluar");
        System.out.print("Pilihan: ");
    }
    //Method push: nambahin website ke tumpukan (1)
    public static void
    kunjungiWebsite_2511532015(Stack<Website_2511532015>
    history, Scanner input) {
        System.out.print("Masukkan Judul: ");
        String judulInput = input.nextLine();
        System.out.print("Masukkan URL: ");
        String urlInput = input.nextLine();

        history.push(new Website_2511532015(judulInput,
        urlInput));
        System.out.println("Berhasil mengunjungi halaman!");
    }
    //Method pop: hapus elemen teratas buat tombol back (2)
    public static void
    tombolBack_2511532015(Stack<Website_2511532015> history) {
        if (history.isEmpty()) {
            System.out.println("History kosong");
        } else {
            Website_2511532015 halamanKeluar = history.pop();
            System.out.println("Kembali dari halaman:");
            System.out.println(halamanKeluar);
        }
    }
    //Method peek: lihat halaman yang aktif (3)
    public static void
    lihatHalamanAktif_2511532015(Stack<Website_2511532015>
    history) {
        if (history.isEmpty()) {
            System.out.println("Tidak ada halaman yang sedang
            aktif dibuka.");
        } else {
            System.out.println("Halaman Aktif Saat Ini:");
            System.out.println(history.peek());
        }
    }
    //Method untuk cek status history
    public static void cekStatusHistory_2511532015
    (Stack<Website_2511532015> history) {
        if (history.isEmpty()) {
            System.out.println ("History kosong");
        } else {
            System.out.println ("Status history: "+
            history.size()+" halaman");
        }
    }
}

```

```

    }
}

public static void main (String[] args) {
    Stack<Website_2511532015> history = new Stack<>();
    Scanner input = new Scanner (System.in);
    int pilihan;

    do {
        tampilkanMenu_2511532015();
        pilihan = input.nextInt();
        input.nextLine();

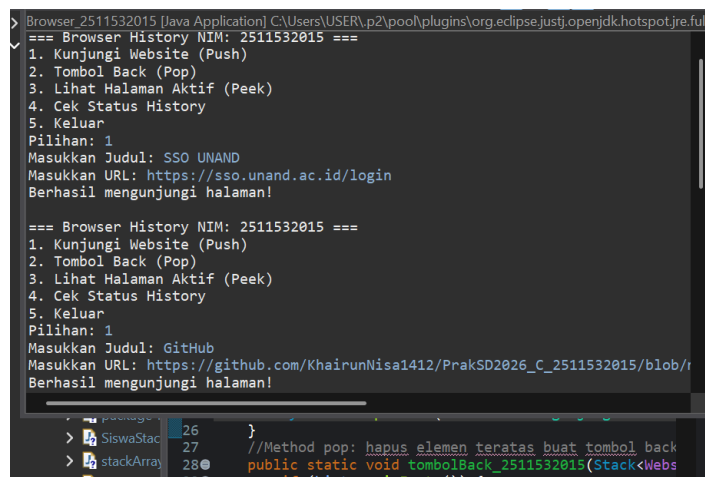
        switch (pilihan) {
            case 1:
                kunjungiWebsite_2511532015(history, input);
                break;
            case 2:
                tombolBack_2511532015(history);
                break;
            case 3:
                lihatHalamanAktif_2511532015(history);
                break;
            case 4:
                cekStatusHistory_2511532015(history);
                break;
            case 5:
                System.out.println("\nTerima kasih telah
menggunakan Browser History");
                break;
            default:
                System.out.println("Pilihan tidak valid, silakan
coba lagi!");
        }
    } while (pilihan != 5);
    input.close();
}
}

```

Kelas Browser berfungsi sebagai pengendali utama program yang mengelola simulasi riwayat penjelajahan web menggunakan struktur data Stack dengan prinsip LIFO (Last In First Out). Kelas ini mendeklarasikan objek `Stack<Website>` sebagai wadah penyimpanan dinamis yang memastikan halaman web yang terakhir dikunjungi selalu berada di posisi teratas tumpukan, sehingga menjadi halaman pertama yang akan diakses saat pengguna menekan tombol kembali. Setiap operasi utama seperti mengunjungi halaman baru, kembali ke halaman sebelumnya, melihat halaman aktif, dan mengecek status riwayat diimplementasikan dalam method-method terpisah yang memanfaatkan method bawaan Stack yaitu

push() untuk menambahkan data, pop() untuk menghapus dan mengembalikan elemen teratas, peek() untuk mengintip elemen teratas tanpa menghapusnya, serta isEmpty() untuk memvalidasi kondisi tumpukan.

3. Penjelasan output kelas Browser_2511532015

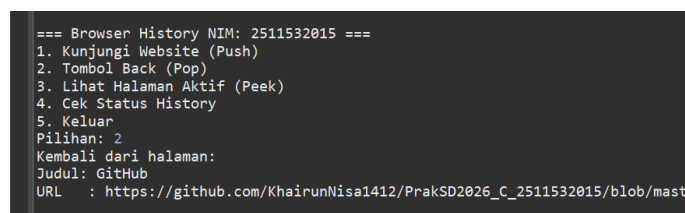


```
Browser_2511532015 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full
=== Browser History NIM: 2511532015 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511532015 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: GitHub
Masukkan URL: https://github.com/KhairunNisa1412/PrakSD2026_C_2511532015/blob/...
Berhasil mengunjungi halaman!

//Method pop: hapus elemen teratas buat tombol back
public static void tombolBack_2511532015(Stack<Web>
if (history.isEmpty()) {
```

Ketika pengguna memilih opsi pertama, program akan meminta input judul dan URL, kemudian mengonversinya menjadi objek baru yang langsung ditambahkan ke puncak tumpukan melalui method push, disertai pesan konfirmasi bahwa halaman berhasil dikunjungi.



```
=== Browser History NIM: 2511532015 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 2
Kembali dari halaman:
Judul: GitHub
URL : https://github.com/KhairunNisa1412/PrakSD2026_C_2511532015/blob/mast
```

Pada pilihan kedua, sistem melakukan pengecekan awal terhadap kondisi tumpukan; jika tidak kosong, method pop akan mengambil sekaligus menghapus halaman teratas sesuai prinsip LIFO, lalu menampilkan detail judul dan URL halaman yang baru saja ditinggalkan, sedangkan jika tumpukan kosong akan muncul peringatan khusus.

```

=== Browser History NIM: 2511532015 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 3
Halaman Aktif Saat Ini:
Judul: SSO UNAND
URL : https://sso.unand.ac.id/login

```

Opsi ketiga memanfaatkan method peek untuk menampilkan informasi halaman yang sedang aktif di posisi teratas tanpa menghapusnya dari riwayat

```

=== Browser History NIM: 2511532015 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
Status history: 1 halaman

```

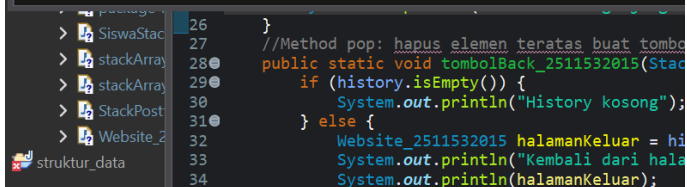
Saat opsi keempat dipilih, program memanggil method size() dan isEmpty() untuk menghitung total entri.

```

=== Browser History NIM: 2511532015 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 5

Terima kasih telah menggunakan Browser History

```



```

26 }
27 //Method pop: hapus elemen teratas buat tombol back
28 public static void tombolBack_2511532015(Stack<Website_2511532015> history) {
29     if (history.isEmpty()) {
30         System.out.println("History kosong");
31     } else {
32         Website_2511532015 halamanKeluar = history.pop();
33         System.out.println("Kembali dari halaman: " + halamanKeluar.getJudul());
34         System.out.println(halamanKeluar.getURL());

```

Terakhir, pilihan keluar akan menghentikan perulangan menu utama dan menampilkan pesan penutup.